الجمهورية الإسلامية الموريتانية
شرف - إخاء - عدل
RÉPUBLIQUE ISLAMIQUE DE MAURITANIE
Honneur - Fraternité - Justice

وزارة التعليم العالي والبحث العلمي
المعهد العالي للرقمنة
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Institut Supérieur du Numérique

Comité d'organisation du championnat « SupNum Coding Challenge Championship' 1447 (S3C'1447)

Sujet du défi N° 1 : Contrubution à la résolution du problème RCPSP	Début : Samedi 20/02/2026 à 12H00 Fin : Vendredi 27/02/2026 à 23H59min

I. Objectif du défi :

Ce défi vise à introduire les participants à un problème scientifique majeur en optimisation combinatoire et à leur permettre de contribuer à l'avancement de l'état de l'art.

L'objectif est de concevoir et implémenter une méthode capable d'améliorer les meilleurs résultats connus pour certaines instances ouvertes du problème RCPSP.

Les participants pourront :

- trouver une solution réalisable avec un makespan inférieur aux meilleurs résultats connus,
- trouver la solution optimale d'une instance ouverte,
- améliorer une borne inférieure existante,
- ou proposer une nouvelle méthode de résolution performante.

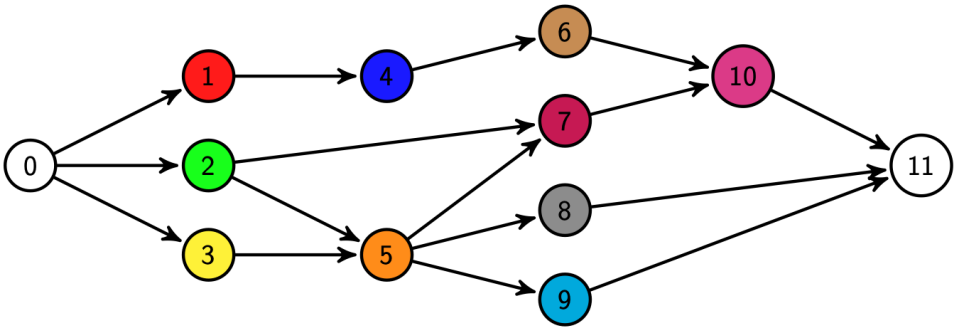
Ce défi constitue une véritable contribution scientifique potentielle.

II. Probléme d'ordonnancement de projet à contraintes de ressources limités (Resource Constrained Project Scheduling Problem (RCPSP))

1. Description du problème

Il s'agit d'un projet découpé en n tâches. Chque tâche i a une durée pi. Il existent de rélations de précdences entre les taches (Certaines tâches ne peuvent pas commencer avant la fin d'autres). Pour la réalisaiton des tâches on dispose d'un ensemble de resources R. Chaque resource k de R a une capacité maximum B_k que l'on peut pas dépasser. pour chaque k de R et pour chaque i de n, b_{ik} est le besoin de la tache i en resource k. L'objectif est de trouver une ordonnancement (date de debut de chaque tache) qui respecte les contraintes suivates : chaque tache doit être executée pendant toute sa durée et sans interruption. Pendant l'éxecution d'une tache, tous ces besoins en resources doivent être disponibles. Une resource ne peut pas dépasser sa capacité à acun moment. Une tâche ne peut commencer avant la fin de toutes ses predecesseurs. Les resources sont renovelables et cumulatives (la capacite se renouvelle à la fin de son utilsation par une tâche et la resource peut faire plusieurs tâches parrallement dans la limite de sa capacité)

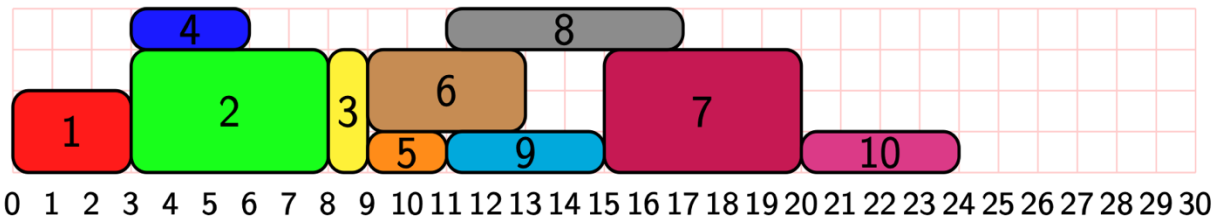
- R ensemble de ressources, disponibilité limitée $B_k \geq 0, k \in R$,
- A ensemble d'activités (tâches), durée $p_i \geq 0, i \in A$, demande $b_{ik} \geq 0$ pour $k \in R$,
- E ensemble de contraintes de précédence (i, j), $i, j \in A$, $i < j$
- $\mathcal{T} = [0, T]$ intervalle de temps (horizon d'ordonnancement)



$|R| = 1, B = 4, \mathcal{T} = [0, 30]$

i	p_i	b_i
1	3	2
2	5	3
3	1	3
4	3	1
5	2	1
6	4	2
7	5	3
8	6	1
9	4	1
10	4	1

Solution de l'exemple :



La solution dans le diagramme ci-dessus est reqlisqble et donne un Cmax = 24 c'est-à-dire que le projet est realise dans 24 perdiodes de temps en respectant toutes les contrqintes (duree, capcite des resources, besoin des taches en ressources, relations de precedences)

Pour résumer, le Resource-Constrained Project Scheduling Problem (RCPSP) est un problème d'ordonnancement de projet défini par :

- un ensemble de n activités
- chaque activité i possède :
  - une durée p_i
  - une consommation b_{ik} pour chaque ressource k
- un ensemble de ressources renouvelables R
- chaque ressource k possède une capacité maximale B_k
- des contraintes de précédence entre activités

Une solution valide doit respecter :

1. Chaque activité est exécutée sans interruption
2. Une activité ne peut commencer qu'après la fin de tous ses prédécesseurs
3. À chaque instant, la consommation totale de chaque ressource ne doit pas dépasser sa capacité
4. Les ressources sont renouvelables

L'objectif est de Trouver les dates de début des activités minimisant : Cmax = durée totale du projet (makespan)

2. Methdodes de resolution du RCPSP

Le RCPSP est un problème d'optimisation combinatoire situé au cœur de nombreuses applications : Gestion de projet, gestion de production, génie des procédés, architectures de processeurs, ...

En terme de complexité, le RCPSP est un problème NP-difficile qui pose un défi computationnel depuis les années 60

Comme tout problème NP-Difficile, la résolution du RCPSP a fait l'objet de beaucoup de recherche afin de trouver des méthodes efficaces . Deux catégories de méthodes existent :

a) **Méthodes exactes** : Les méthodes exactes permettent à trouver la solution optimale d'une instance donnée. Cependant, ils prennent beaucoup de calcul ce qui ne permet pas à résoudre des instances de taille important avec ce genre d'approches. Parmi, les méthodes exactes, on trouve :

- Programmation linéaire en nombres entiers (PLNE)
- Méthodes de séparation et évaluation (Branch & Boundd)
- Programmation par contraintes (CP)

b) Méthodes approchées :

Vues les limites des méthodes exactes, d'autres techniques de résolution approchées ont été proposés par plusieurs spécialistes. Ces méthodes approchées, ont l'avantage de pouvoir trouver des solutions réalisables en temps raisonnable, même pour les instances de grandes tailles. Cependant, elles ne garantissent pas l'optimalité des solutions trouvé.

Parmi, les méthodes approchées, on peut citer :

- Heuristiques spécifiques
- Metaheuristiques [Kolisch & Hartmann 2006, A. & Rivreau 2008] : comme les algorithms génétiques, recherche tabou, etc
- D'autres approches combinanat plusieurs technqiues existeent comme, par exemples, les mathereustiques (Programmation lineaire + heuristiques ou métaheuristiques)
- Les techniques de Machine learning ont egalement été employées récemment par les chercheurs pour la sélection des meilleurs algorithmes, etc.

c) **Bornes inférieures :** d'autres études se sont interésé à proposer des méthdes capables de trouver une durée minimale du projet (Borne inférieure) sans résoudre l'instance. Ces technqiues aident à

- Evaluer les méthdoes approchées (Mesurer la deviation moyenne des solutions trovées par une méthode par rapport aux valeurs de la borne inférieure calculées par une méthode de clacul de BI).
- Accélérer les approches de type branch & bound en utilsiant les bornes inférieures afin de pouvoir elager les nœuds rapidement.

3. **Jeux de données (data sets) et benchmarks :** plusieurs jeux de données ont étés préparés et utilisés dans la littérature afin de comparer les différentes méthodes. L'un des jeux de données les plus utilisés est celui du PSPLIB (https://www.om-db.wi.tum.de/psplib/data.html)

III. Travail demandé

On vous demande de :

1. Implementer la méthode propsée dans l'artice 1 (fichier joint). Tester cette méthode en

vérifiant que vous aviez obtenu les résultats annoncés dans le papier

2. Télecharger les instances de 60 tâches du RCPSP single mode et les fichiers qui contiennent leur solutions.
   - Le data set : j60.sm.tgz ou j60.sm.zip
   - Les solutions: j60hrs.sm
3. Tester la méthode de l'article 1 sur les instances des 60 tâches dont la solution optimale n'est pas encore connue selon le fichier de solution (LB < UB)
4. Défier la communoté : votre misson est maintenant d'améliorer les résultat sur les instances de 60 tâches. Pour se faire, vous deviez analysez les algorithmes de résolution (Exactes ou approchées) et/ou des algorithmes de calcul des bornes inférieures qui existent déja. Puis, vous essayez d'apporter une amélioration sur l'une de ces méthodes ou proposez une nouvelle méthode. Vous implémentez la nouvlle méthode et la tester sur les instances non encore résolues de 60 tâches.
5. Comaraison des solutions trouvées avec les solutions dans les fichiers de PSPLIB. Si vous trouverez une solution méillure que ce qui est déjà dans les fichiers, vous la reportez dans un fichier Word selon le foramt du tableau ci-dessous

Instance	Type Solution(LB/OPT/HEUR	Valeur	Techniaue de révolution	Temps d'exécution	Caracteristique de la machine utilisée	Ancienne Valeur

IV. Rendu à soumettre

- Code source + les résultats (fichier texte, word ou excel) de l'algorithme d'article fourni au défi.
- Résultat la méhode dans l'article 1 sur les instances non résolues de 60 tâches
- Document Word ou powerPoint expliquant le travail réalisé par le groupe pour contrubuer à l'amélioration des résultats
- Code source des algorithmes implémentés dans le but de trouver des meilleures résultas par rapport à résultats existants
- En cas d'obtention des résultats améliorant l'existant, vous soumetez le fichier Word demandé dans la section précedente (Point 4)

V. Evaluation

- En cas d'obtention des noveaux résultats : 100/100 points, un prix spécial vous sera décerné
- En cas de non obtention de noveau résultats :
  - Un jury examinera les solutions proposées et attrubuera une note sur la base du travail et effort fourni par le groupe

ANNEXE : Qeules codes sources trouvés sur internet qui peuvent être utiles

Type de méthodes	**Description**	**Code source**
Méthodes exactes – Programmation linéaire entière (MILP)	Modélise le RCPSP comme un problème d'optimisation avec variables binaires et contraintes linéaires.	https://github.com/AlkisPlas/MPRJ_RCPSP
Programmation par contraintes (CP)	Utilise la propagation de contraintes et le backtracking intelligent.	https://github.com/ptal/kobe-scheduling
Métaheuristiques	Utilise une population de solutions avec opérateurs de mutation et crossover.	https://github.com/hww-dev/RCPSP_Genetic_Algorithm
Machine Learning – Methodes	Apprend une politique de scheduling à partir de la structure du graphe du projet.	https://github.com/jolibrain/wheatley